

SPIDriver on the RP2040

A Portable-C Port of the Original 8051 Firmware

firmware-c port

August 15, 2026

Abstract

The SPIDriver is a host-controlled SPI debug tool: a PC sends byte-stream commands over a serial link, the device clocks them out on an SPI bus, and a small LCD renders the resulting bus traffic as a six-channel logic-analyzer display. The original firmware was written in MyForth for the Silicon Labs EFM8BB10 (8051 core). This document describes the portable-C port targeting the Raspberry Pi Pico (RP2040): a hardware-abstracted core, a hardware-SPI HAL, a TinyUSB CDC transport, and an ST7735S display driver.

Contents

1	Architecture overview	1
2	Pin map	2
3	USB CDC transport (TinyUSB)	3
3.1	Two correctness-critical details	3
3.2	1200-baud re-flash	3
3.3	RP2040 USB errata workaround	3
4	Host protocol	4
5	CRC-16	4
6	Measurements	4
7	Main loop and concurrency	5
8	Logic-analyzer display	5
8.1	ST7735S driver	5
8.2	Fonts	6
9	Build	6

1 Architecture overview

The port is split into three layers, shown in Figure 1. The portable core (`src/`) contains all protocol, measurement, and display logic and *never touches hardware directly*; every hardware access goes through the `spidriver_hal` function table. Target ports implement that table. Two targets exist: `pico/` for the RP2040 and `host/` for a POSIX test harness.

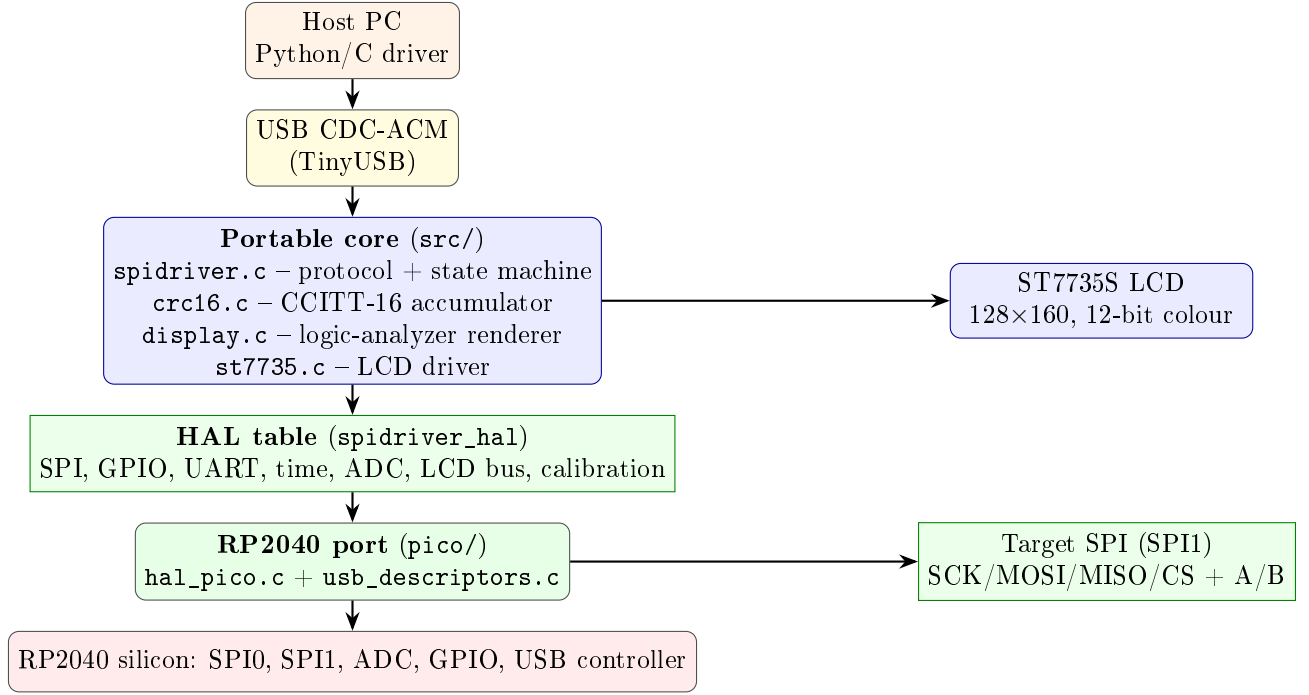


Figure 1: Layered architecture. The core is target-independent; only the HAL knows the RP2040.

2 Pin map

Both SPI buses are *hardware* SPI (no bit-banging). The LCD rides SPI0; the device-under-test rides SPI1. RP2040 hardware-SPI pins are fixed per-function sets, which forces one compromise: target MOSI must land on GP11 (physical pin 15) rather than the original board’s pin 13.

Function	GPIO	Physical pin	Peripheral
LCD SCK	GP2	4	SPI0 SCK
LCD MOSI	GP3	5	SPI0 TX
LCD DC	GP4	6	GPIO out
LCD CS	GP5	7	GPIO out
Target SCK	GP10	13	SPI1 SCK
Target MOSI	GP11	15	SPI1 TX
Target MISO	GP8	11	SPI1 RX
Target CS	GP9	12	GPIO out
Signal A	GP6	9	GPIO out
Signal B	GP7	10	GPIO out
VBUS sense	GP26	–	ADC0
Current sense	GP27	–	ADC1
Temperature	–	–	internal ADC4

Table 1: RP2040 pin assignment. LCD RESET is tied to 3V3, as on the original board.

The clock frequencies mirror the original: the LCD runs at 16 MHz, the target bus at 500 kHz (the EFM8 clocked ≈ 490 kHz). Both are configurable via `SPI_TARGET_HZ`.

3 USB CDC transport (TinyUSB)

The original board used an FTDI UART bridge; the RP2040 replaces it with its native USB controller and TinyUSB in device mode. The device enumerates as a single CDC-ACM serial port.

Parameter	Value
VID/PID	2E8A:000A
Class	CDC-ACM (two interfaces: control + data)
Control EP	0x81 (8-byte)
Bulk OUT	0x02 (64-byte)
Bulk IN	0x82 (64-byte)
Serial string	derived from <code>pico_get_unique_board_id()</code>
RX/TX FIFO	256 bytes each

Table 2: USB descriptor summary.

3.1 Two correctness-critical details

Two bugs surfaced during bring-up and shaped the final code:

1. **Interface count.** The configuration descriptor initially declared one interface, so the host never probed the CDC *data* interface and the bulk endpoints were never instantiated. The fix is `USBD_ITF_MAX = 2` (control + data) inside `usb_descriptors.c`.
2. **Explicit flush.** The bundled TinyUSB only auto-flushes the TX FIFO once 64 bytes are queued. Single-byte replies would sit in the FIFO indefinitely, so `uart_put()` calls `tud_cdc_write_flush()` after every byte.

3.2 1200-baud re-flash

Opening the port at 1200 baud reboots the chip into the RP2040 bootrom (BOOTSEL), the same convention as `pico_stdio_usb` and Arduino:

```
1 void tud_cdc_line_coding_cb(uint8_t itf, cdc_line_coding_t const *line_coding) {
2     (void)itf;
3     if (line_coding->bit_rate == 1200) {
4         reset_usb_boot(0, 0);
5     }
6 }
```

This links `pico_bootrom` and makes remote flashing button-free: `stty -F /dev/ttyACM0 1200` drops the Pico into the mass-storage bootrom, and a `.uf2` is copied in.

3.3 RP2040 USB errata workaround

The bundled TinyUSB gates a bulk-IN workaround for RP2040 errata E5/E15 behind `TUD_OPT_RP2040_USB_DEVICE_UFRAME_FIX` which no one defines by default. Without it, an armed bulk-IN buffer can stop answering IN tokens and the tail of a reply never reaches the host. The port enables it in `tusb_config.h`:

```
1 #define TUD_OPT_RP2040_USB_DEVICE_UFRAME_FIX 1
```

Command	Bytes	Effect
?	1	transmit 80-byte status line
e 	2	echo
s	1	select (CS low)
u	1	deselect (CS high)
a 	2	set A from bit 0 of
b 	2	set B from bit 0 of
x	1	disconnect (tri-state target bus)
0x80-0xBF	1 + n	duplex: clock n bytes, echo n MISO bytes
0xC0-0xFF	1 + n	write-only: clock n bytes

Table 3: Command set. Transfer length $n = (\text{cmd} \& 0x3F) + 1$.

4 Host protocol

The byte-stream protocol is drop-in compatible with the original drivers. Bytes are framed by count, not delimiters.

The status line is *exactly* 80 bytes — the host driver’s `getstatus` slices it blindly:

```
[spidriver1 5303284740C3781C 48 1.928 322 31.2 0 0 1 0000]
```

Fields: product, serial, uptime s, Vbus V, current mA, temperature °C, A, B, CS, CRC-16.

5 CRC-16

The CRC is the CCITT-16 *XMODEM* accumulator (polynomial 0x1021, initial value 0). Every byte clocked out and every MISO byte clocked in is folded into the accumulator. The status line reports the *bit-reversed* accumulator to match the original firmware:

```
1 uint16_t crc = crc16_bitreverse(st.crc); // status reports this
```

6 Measurements

Three channels are sampled: Vbus (ADC0, via a divider), target current (ADC1, via the ZXCT1110 current mirror on the original board), and the internal temperature sensor (ADC4).

Each channel is read four times and box-averaged, then folded through an exponential moving average with $\alpha = 18$, replicating the original `ema` word:

$$new = \frac{7 \cdot old + sample}{8} \quad (1)$$

All arithmetic is in “raw16” units, where $raw16 = raw12 \ll 4$ (a 12-bit ADC read left-shifted to 16 bits). Calibration constants are scaled to raw16:

Constant	Value	Meaning
cal_vbus_mv	8250	3.3 V ref, divider $V_{bus} = 2.5 V_{pin}$
cal_current_ma	1375	ZXCT1110, $R_s=0.1 \Omega$, $R_g=2.4k$
cal_current_zero	0	offset subtracted before scaling
cal_temp_coef	19173	RP2040 temp-sensor slope
cal_temp_offset	4372	deci-°C at raw16 = 0

Table 4: Calibration constants (raw16 scale).

7 Main loop and concurrency

Concurrency is deliberately simple. A 1ms repeating hardware timer fires `spidriver_tick()`, which advances the uptime counter and decrements the slow-refresh counter. All other work happens in the foreground loop:

```
1 void spidriver_mainloop(spidriver_hal *hal) {
2     for (;;) {
3         conversions(hal);                // ADC + EMA + scaling
4         for (int i = 0; i < 64; i++) {    // drain command queue
5             if (!spidriver_service(hal)) break;
6         }
7         if (st.dirty) {                  // waveform redraw
8             st.dirty = false;
9             display_waves(&st.lcd, st.history, st.hist_count, st.port);
10        }
11        if (st.slowc == 0) {              // 250 ms reading refresh
12            st.slowc = 250;
13            display_results(&st.lcd, st.vbus_mv, st.cur_ma);
14        }
15    }
16 }
```

The loop mirrors the original firmware’s split between a background measurement “thread” and a foreground command service.

8 Logic-analyzer display

The display renders six horizontal traces: SCK, MISO, MOSI, CS, A, B. A 14-entry history ring (the original’s 42-byte ring) stores the most recent bus events. Each event is tagged with a *code* byte:

Code	Meaning	Width (columns)
'a'	port change (CS/A/B)	18
'b'	write-only byte	18
'c'	duplex byte (MOSI→MISO)	18
'd'	disconnect	8

Table 5: Waveform entry codes and column widths.

Rendering walks the history newest-first, right-to-left. Two key state machines reconstruct the analogue signal:

- **MISO/MOSI** track a *previous level* across byte boundaries, drawing a vertical edge whenever the last bit of one byte differs from the first bit of the next.
- **CS/A/B** track the port state *backward* through history: each 'a' entry stores the port value *before* the change, so walking right-to-left reconstructs the true signal level during every transfer.

Each byte is drawn as eight 2-pixel-wide bit cells (16 columns) plus a trailing return-to-idle column, matching the original’s 18-column byte cell.

8.1 ST7735S driver

The panel is 128×160 in 12-bit colour. Two details matter:

1. **12-bit packing.** Pixels are 4-4-4 (BGR order) and packed two-per-three-bytes:

$$\underbrace{(b_0 \ll 4 | g_0)}_{\text{byte0}} \quad \underbrace{(r_0 \ll 4 | b_1)}_{\text{byte1}} \quad \underbrace{(g_1 \ll 4 | r_1)}_{\text{byte2}} \quad (2)$$

The original naive port sent 16 bits per pixel, scrambling colours; the fix packs to three bytes per two pixels.

2. **Orientation.** Everything renders in literal portrait coordinates with `MADCTL = 0x08` (BGR). The original split text (0xC8) and waveform (0xA8) orientations; this port unifies them.

8.2 Fonts

Two fonts are embedded, generated by `tools/genfonts.py`:

- **Micro** — 4×5 hexadecimal glyphs for the byte annotations under MISO/MOSI, nibble-packed (2 px/byte).
- **Big** — IBM Plex Sans SemiBold at 13 pt, regenerated from the original face, 4-bit greyscale, nibble-packed. Digits are fixed 8×9. The generator crops glyphs from $y = 5$ (the ink start for 13 pt Plex Sans); the original port cropped from $y = 0$, which captured five blank rows and truncated the digit bottoms.

9 Build

```
export PICO_SDK_PATH=/path/to/pico-sdk
cmake -B build
cmake --build build -j
# flash:
stty -F /dev/ttyACM0 1200          # reboot into BOOTSEL
udisksctl mount -b /dev/sdX1
cp build/spidriver_pico.uf2 /media/\$USER/RPI-RP2/
```

The core (`src/`) is shared with the POSIX host test target (`host/`), so the protocol can be validated on a desktop without hardware: `printf '??' | ./host/build/spidriver_host` returns the 80-byte status line. A Python regression harness (`host/test_spidriver.py`) exercises status, echo, CS/A/B control, duplex and write-only transfers, and a 100-round stress test over the real USB link — including full-raw tty handling (canonical mode and XON/XOFF would otherwise swallow reply bytes).